Operating System – Unit I

Operating System Services and Components

MSBTE K-Scheme | Diploma CO Branch | Third Year (TY)

1.1 Operating System: Concept and Functions

Concept

An **Operating System (OS)** is system software that acts as an **intermediary between the computer user and the computer hardware**. It manages hardware resources and provides a platform on which application programs can run.

Think of the OS as a **resource manager** and **traffic controller**: - It controls and coordinates the use of hardware among various application programs. - It provides a convenient, user-friendly environment so users don't need to understand hardware details. - It ensures the computer system is used efficiently.

Layered view of a computer system:

```
       Users
         |
Application Programs
         |
Operating System   <-- manages everything below
         |
  Computer Hardware
```

Definition

"An Operating System is a program that acts as an interface between the user and computer hardware, and controls the execution of all kinds of programs."

Functions of Operating System

1. **Process Management**
   - Creating, scheduling, suspending, resuming, and terminating processes.
   - Handling process synchronization and communication.
2. **Memory Management**
   - Keeping track of which part of memory is in use and by whom.
   - Allocating and deallocating memory space as needed.
3. **File Management**
   - Creation, deletion, and manipulation of files and directories.
   - Mapping files onto secondary storage and controlling access.
4. **Device/I-O Management**
   - Managing I/O devices through device drivers.
   - Providing a uniform interface to access hardware devices.

5. **Secondary Storage Management**
   - Managing free space, storage allocation, and disk scheduling.
6. **Security and Protection**
   - Protecting data and resources from unauthorized access.
   - User authentication (login/password mechanisms).
7. **Networking**
   - Managing communication between distributed/networked systems.
8. **Command Interpretation**
   - Provides interface (CLI/GUI) for the user to interact with the system.
9. **Error Detection**
   - Constantly monitors the system to detect errors and avoid system malfunction.
10. **Resource Allocation**
    - Allocates resources (CPU, memory, I/O devices) to multiple users/jobs efficiently.
11. **Job Accounting**
    - Keeps track of time and resources used by various jobs/users.

1.2 Different Types of Operating System

a) Batch Operating System

- Jobs with similar needs are grouped (batched) and executed together without user interaction.
- An operator collects jobs and executes them one by one; output is given later.
- **No direct interaction** between user and computer during execution.

Advantages: Increases efficiency, reduces idle CPU time, good for repetitive jobs. **Disadvantages:** Difficult to debug, no interaction, CPU may sit idle waiting for I/O, starvation possible for long jobs. **Example:** Payroll systems, bank statement generation.

b) Multiprogrammed Operating System

- Multiple programs reside in main memory at the same time; CPU switches between them.
- When one program waits for I/O, CPU executes another → **maximizes CPU utilization**.
- Uses **job scheduling** to decide which job enters memory.

Advantages: High CPU utilization, better throughput. **Disadvantages:** CPU scheduling is complex; requires memory management.

c) Time-Shared (Multitasking) Operating System

- An extension of multiprogramming.
- CPU switches between multiple jobs so frequently that users feel they are getting immediate response (**illusion of dedicated system**).
- Uses **time slices (quantum)** for each process.

Advantages: Quick response time, reduces CPU idle time, supports multiple users simultaneously. **Disadvantages:** Complex scheduling, needs high security due to shared data, communication overhead. **Example:** UNIX.

d) Multiprocessor System

- System has **more than one CPU/processor** sharing common memory and peripherals.
- Also called **parallel systems** or **tightly coupled systems**.

Types: - **Symmetric Multiprocessing (SMP):** Every processor performs all tasks equally. - **Asymmetric Multiprocessing (AMP):** One master processor controls the system; others (slaves) execute assigned tasks.

Advantages: Increased throughput, economical (shared resources), increased reliability (if one processor fails, others continue – graceful degradation). **Disadvantages:** Complex OS design required.

e) Distributed Operating System

- Manages a group of **independent computers connected via a network** and makes them appear as a single system to the user.
- Also called **loosely coupled systems** — each processor has its own local memory.

Advantages: Resource sharing, faster computation, reliability (failure of one node doesn't affect the whole system), reduced load on host. **Disadvantages:** Requires robust network, security is harder to maintain, complex software needed. **Example:** LOCUS.

f) Real-Time Operating System (RTOS)

- Provides a response within a **strict, fixed time constraint** (deadline).
- Used where timing is critical.

Types: - **Hard Real-Time System:** Must meet the deadline strictly (e.g., missile guidance, airbag systems) — missing deadline = system failure. - **Soft Real-Time System:** Deadline is desirable but not mandatory (e.g., video streaming, online reservation systems).

Advantages: Maximum consistency, error-free, ideal for embedded applications. **Disadvantages:** Limited multitasking, expensive, complex algorithms.

g) Mobile OS (Android OS)

- Designed specifically for handheld devices (smartphones, tablets).
- **Android OS** is a Linux-kernel-based, open-source OS developed by Google.

Android Architecture (layers): 1. Linux Kernel (core system services – drivers, memory, power management) 2. Hardware Abstraction Layer (HAL) 3. Android Runtime (ART) & Native C/C++ Libraries 4. Java API Framework 5. System Apps (Applications layer)

Features: Touch-screen support, multitasking, connectivity (Wi-Fi, Bluetooth, GPS), Google Play Store for apps, open-source and customizable.

1.3 Command Line Based vs GUI Based Operating Systems

Command Line Interface (CLI) Based OS

DOS (Disk Operating System)

- Single-user, single-tasking, command-line based OS by Microsoft.
- User types commands (e.g., `DIR`, `COPY`, `DEL`) at a prompt (`C:\>`).
- **Features:** Simple, requires less memory, no multitasking/multi-user support, no GUI.
- **Limitations:** No protection between programs, limited memory addressing (up to 1MB conventional memory), single user only.

UNIX

- Multi-user, multitasking, portable OS developed at Bell Labs.
- Written mostly in C language, making it portable across hardware platforms.
- **Features:** Hierarchical file system, powerful shell (command interpreter), supports multiple users/processes simultaneously, strong security and file permission system (owner/group/others – read/write/execute).
- **Components:** Kernel, Shell, File System, Utilities/Commands.

GUI (Graphical User Interface) Based OS

Windows

- Developed by Microsoft; uses icons, windows, menus, and pointers (WIMP interface).
- Multi-user (with accounts), multitasking OS.
- **Features:** User-friendly, Plug and Play support, wide software/hardware compatibility, built-in utilities (File Explorer, Control Panel), supports networking.

Linux

- Open-source, Unix-like OS based on the Linux kernel created by Linus Torvalds.
- Available in different **distributions** (distros): Ubuntu, Fedora, Red Hat, Debian, etc.
- **Features:** Free and open-source, highly stable and secure, multiuser & multitasking, customizable, strong community support, supports both CLI (terminal/shell) and GUI (desktop environments like GNOME, KDE).

Mac OS

- Developed by Apple Inc. for Macintosh computers; Unix-based.
- **Features:** Elegant and intuitive GUI, tight hardware-software integration, high security, seamless integration with other Apple devices (iCloud, Handoff), stable performance.

1.4 Different Services of Operating System & System Calls

Services Provided by OS

1. **Program Execution** – Loading a program into memory and running it, handling normal and abnormal termination.
2. **I/O Operations** – Since user programs can’t directly access I/O devices, OS provides the means to perform I/O.
3. **File System Manipulation** – Programs need to read/write/create/delete files; OS manages permissions too.

4. **Communication** – Exchange of information between processes, either on the same computer or over a network (via shared memory or message passing).
5. **Error Detection** – OS constantly checks for possible errors in CPU, memory hardware, I/O devices, or user programs.
6. **Resource Allocation** – When multiple jobs run concurrently, resources (CPU cycles, memory, storage) must be allocated to each.
7. **Accounting** – Keeps track of which users use how many/which kind of resources.
8. **Protection and Security** – Ensures controlled access to resources/data, especially in multi-user systems; protects against unauthorized access.

System Calls

Concept

A **system call** is the programming interface through which a **user program requests a service from the operating system's kernel**. It is the only legitimate entry point into the kernel.

- User programs run in **user mode** (restricted); to perform privileged operations (like accessing hardware), they must switch to **kernel mode** via a system call.
- System calls are usually invoked through a high-level Application Program Interface (API) rather than direct use.

Working (steps): 1. User program executes a system call instruction (trap). 2. Control transfers from user mode to kernel mode. 3. OS kernel executes the requested service. 4. Control returns to the user program along with the result.

```
User Program → System Call → Trap to Kernel → OS executes service → Return to User Program
```

Types of System Calls

1. **Process Control**
   - Create, terminate, load, execute, abort, end process
   - `fork()`, `exit()`, `wait()`, `exec()`
2. **File Management**
   - Create, delete, open, close, read, write, reposition file
   - `open()`, `read()`, `write()`, `close()`
3. **Device Management**
   - Request/release a device, read, write, reposition, get/set device attributes
   - `ioctl()`, `read()`, `write()`
4. **Information Maintenance**
   - Get/set time, date, system data, process/file/device attributes
   - `getpid()`, `alarm()`, `sleep()`
5. **Communication**
   - Create/delete communication connection, send/receive messages, transfer status information
   - `pipe()`, `shmget()`, `send()`, `receive()`
6. **Protection** (sometimes included as a 6th category)
   - Control access to resources, get/set permissions
   - `chmod()`, `chown()`

1.5 Operating System Components

The OS is logically organized into several manager components, each responsible for a specific resource.

1. Process Management

- A **process** is a program in execution.
- Responsibilities:
  - Creating and deleting processes (user and system)
  - Suspending and resuming processes
  - Providing mechanisms for **process synchronization**
  - Providing mechanisms for **process communication** (IPC)
  - Providing mechanisms for **deadlock handling**
  - **CPU Scheduling** – deciding which process runs next

2. Main Memory Management

- **Main memory (RAM)** is a large array of bytes shared by CPU and I/O devices; it is volatile.
- Responsibilities:
  - Keeping track of which parts of memory are currently used and by whom
  - Deciding which processes/data to move in and out of memory
  - Allocating and deallocating memory space as needed
  - Techniques used: Paging, Segmentation, Partitioning, Virtual Memory

3. File Management

- A **file** is a collection of related information defined by its creator.
- Responsibilities:
  - Creating and deleting files and directories
  - Supporting primitives for manipulating files/directories
  - Mapping files onto secondary storage
  - Backing up files on stable (non-volatile) storage media
  - Maintaining file access permissions/protection

4. I/O (Input-Output) Management

- One purpose of an OS is to **hide peculiarities of hardware devices** from the user.
- Responsibilities:
  - A general device-driver interface
  - Drivers for specific hardware devices (keyboard, mouse, printer, disk, etc.)
  - Buffer/cache management for I/O
  - Spooling (managing jobs for devices like printers)

5. Secondary Storage Management

- Since main memory is volatile and small, the computer system needs **secondary storage** (hard disk, SSD) to back it up.
- Responsibilities:
  - **Free space management** – tracking unused disk blocks
  - **Storage allocation** – deciding how storage is allocated to files (contiguous, linked, indexed)
  - **Disk scheduling** – deciding order of disk I/O requests (FCFS, SSTF, SCAN, C-SCAN, etc.) to optimize access time

---

Quick Revision Table

Component	Main Job
Process Management	Handles creation, scheduling, execution of processes
Memory Management	Allocates/deallocates main memory
File Management	Handles file creation, deletion, access
I/O Management	Manages device drivers and I/O operations
Secondary Storage Mgmt	Manages disk space allocation and scheduling

OS Type	Key Feature
Batch	Jobs grouped, no user interaction
Multiprogrammed	Multiple jobs in memory, maximizes CPU use
Time-Shared	Quick switching, gives illusion of dedicated system
Multiprocessor	Multiple CPUs, shared memory
Distributed	Independent networked computers, appear as one system
Real-Time	Strict time deadlines (hard/soft)
Mobile (Android)	Linux-based OS for handheld devices

End of Unit I Notes